SIMATS ENGINEERING
Department of Electrical and Electronics Engineering
ASSESSMENT TOOL – 3

Engineer to Excel

Course Code:	ECA1418	Course Title:	Embedded System
CO Assessed:	CO : 1	Assessment:	AT-3
Weightage:		Total Marks:	
CO1 Statement:			
Student Name:	M. Nandhini	Reg. No.:	192472318
Section / Batch:	3rd Year (Slot-B)	Date:	24-07-26

Question Type	Description	Questions	Total Marks
	Short Answer Test	Q - Q	100

Code of Conduct

I M. Nandhini (192472318) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: M. Nandhini

RUBRICS FOR MARKS CALCULATION

Task	(%)	(%)	(%)	(%)	Total Marks (Each -)
					20
				Overall Total ( )	

Faculty In-charge	Course Coordinator	Head of Department
Name: Dr. C.M. ARUN KUMAR	Signature: _____	Signature: _____
Signature: _____		
Date: 24/7/26	Date: _____	Date: _____

M. Nandhini
192472218
CSE-AI
ECA1418
Embedded System
Dr. C M. ARUN KUMAR.

ASSESSMENT TOOL - 3

1 A can gateway node uses Harvard - architecture Cortex - M7 at 480 MHz with 35% branch frequency. Analyze how pipeline stalls and cache miss penalties determine worst-case interrupt latency for CAN frame reception.

Effect of Pipeline Stalls:

Assuming a worst case branch misprediction penalty of 3 cycles:

Pipeline stall delay: $0.35 \times 3 = 1.05$ cycles per instruction.

Time delay: $1.05 \times 2.08 = 2.18$ ns

Thus, branch-related stalls increase interrupt latency.

Effect of Cache Miss:

Instruction Cache miss = 20 cycles

Data cache miss = 15 cycles

Total = 20 + 15 = 35 cycles

Time penalty $\Rightarrow 35 \times 2.08 = 72.8$ ns

Worst-Case Interrupt Latency:

Assume: Interrupt entry overhead = 12 cycles

Pipeline stall = 3 cycles

Cache miss penalty = 35 cycles.

Total cycles $\Rightarrow 12 + 3 + 35 = 50$ cycles

Latency: 50 × 2.08 = 104 ns

Conclusion:

The worst-case interrupt latency is ≈ 104 ns.

* pipeline stalls caused by frequent branch instructions 35% branch
* instruction and data cache misses increase memory access time.
* interrupt entry overhead Processor saves context before executing ISR

2 Tasks A & B share CAN and SPI Flash with opposite acquisition order, reaching a petri Net deadlock. Identify the deadlock marking, prove the violated property, and evaluate resource-ordering as prevention.

Deadlock Marking:

* Task A holds the CAN resource and waits for SPI Flash.
* Task B holds the SPI Flash resource and waits for CAN.
* Both tasks wait forver → Deadlock.

Violated Property:

* Liveness is violated because neither task can proceed
* The Petri Net reaches a deadlock marking where no transition is enabled.

Resource Ordering Prevention:

* Assign a fixed order for resource acquisition.
* Both Task A & B must request resources in the same order.
* This remote circular waiting and Prevents deadlock.

Conclusion: Using a fixed resource-ordering policy prevents the petri Net deadlock and ensures smooth task execution.

factory CAN network (250 kbps, 25 nodes) shows 4% error frames and TEC > 100 on 3 nodes after adding 5 sensor nodes pushing bus load to 87%. Diagnose root cause and redesign scheduling to restore load below 60%.

Root Cause:

- Adding 5 sensor nodes increased the CAN bus load to 87%.
- High traffic caused 4% error frames and TEC > 100 on 3 nodes.

Redesign:

* Reduce the transmission rate of sensor messages, and give higher priority only to important CAN messages.
* Keep the total bus load below 60%.

Result:

* Bus load < 60%.
* Error frames are reduced.
* TEC value decreases.
* CAN network becomes stable.

A CAN smart home (35 nodes, 250 kbps) must guarantee sub-1 ms fire alarm and sub-200 ms lighting response on the same bus. Analyse feasibility and justify priority assignment.

Feasibility Analysis: A 250 kbps CAN network with 35 nodes can meet both response-time requirements if message priorities are assigned correctly. CAN uses priority-based arbitration, where messages with the lowest CAN ID are transmitted first.

Priority Assignment:

* Fire Alarm: Assign the highest priority (lowest CAN ID) to guarantee transmission within 1 ms.

* Lighting Control: Medium priority to ensure less than 200ms response.

* Other smart home devices: Lower priority.

Justification: During bus arbitration, the fire alarm messages always win; ensuring immediate transmission. Lighting messages may wait briefly but still meet the 200ms deadline. Proper priority assignment and controlled bus load ensure reliable communication for all 35 nodes.

5 Luxury Sedan's single 500 Kbps CAN bus (45 ECU's) reaches 95% load during OTA updates, delaying ABS messages by 12ms against a 5ms safety requirement. Evaluate software-only fix feasibility and specify minimum hardware change.

Software-Only Fix: A software-only solution is partially feasible by: Giving ABS messages the highest priority.

- Reducing or postponing OTA update traffic.
- Scheduling OTA data during the idle bus Periods.

Minimum Hardware Change: Add a second CAN bus (or dedicate a CAN bus for safety-critical ECUs such as ABS). This separates OTA traffic from safety messages, reducing bus load and ensuring ABS messages meet the 5ms deadline.

Conclusion:

Software optimization can improve performance, but the minimum reliable hardware change is adding a dedicated second CAN bus for safety-critical communication.